

Architectural Blueprint

Distributed Agentic Red-Teaming Lab: Validating Consensus Contagion via DuckDB Quack

1. The What: System Architecture

This architecture provisions a high-throughput, automated red-teaming harness designed to empirically validate the **Consensus Contagion** exploit against agentic routing nodes. By leveraging a distributed execution model and centralized high-speed telemetry, the lab bypasses traditional single-writer database bottlenecks.

Core Components & Data Flow

- **Orchestration (Dagster):** Manages the pipeline. Generates parameterized payload mutations (varying Context Padding and Status Anchor severity) and queues them for execution.
- **Execution Tier 1 - Brute Force (Local Docker + Ollama):** Dagster spins up containerized targets (e.g., LangGraph/AutoGen nodes). To mitigate token costs, high-volume initial fuzzing is routed against local LLMs (Llama-3/Phi-3).
- **Execution Tier 2 - Validation (Anthropic/OpenAI APIs):** Payloads that successfully trigger a guardrail collapse in Tier 1 are promoted to Tier 2 for validation against production-grade models.
- **Telemetry Capture (DuckDB Quack Server):** Executing containers use the DuckDB Quack RPC protocol to stream parsed intents, inherited context windows, and failure states concurrently to a centralized DuckDB server, eliminating the Postgres staging layer.
- **Analytics Engine (Polars):** Reads directly from the central DuckDB instance to compute the Bypass Success Rate (BSR) and Context Contagion metrics with zero ETL latency.

2. The So What: Strategic Justification

This is not combining tools for the sake of complexity; it is an engineered solution to specific bottlenecks in AI security benchmarking.

- **Solving the Single-Writer Bottleneck:** Previous iterations required Postgres to handle concurrent writes from parallel Docker workers, adding latency and infrastructure overhead. DuckDB's Quack protocol enables direct, concurrent, zero-copy serialization of vector blocks from worker nodes to the central analytics store.
- **Cost Asymmetry Control:** API token burn is the primary constraint of LLM red-teaming. The two-tier routing architecture ensures we only pay API costs for highly optimized, structurally sound payloads.
- **The "Research-Build-Teach" Leverage:** This architecture serves a triple-threat mandate. It yields the empirical data required for the Black Hat/RSA submissions, tests a beta database protocol in a novel data engineering use case, and establishes a reference architecture for LLM orchestration.

3. Pros & Cons Evaluation

Attribute	Pros	Cons
DuckDB Quack	Zero ETL latency; direct vector block serialization; eliminates Postgres infrastructure.	Protocol is in beta (v1.5.2). Breaking changes expected. Native auth is primitive (shared token). Unencrypted over the wire by default.
Tiered LLM Execution	Massive reduction in API token costs; enables statistically significant sample sizes for payload fuzzing.	Requires maintaining parity between local model prompts and API prompts; Tier 1 models may not accurately mimic Opus/GPT-4o guardrails.
Dagster Orchestration	First-class support for parallel execution and dynamic asset generation based on payload success.	High initial setup complexity for dynamic container provisioning.

4. The Now What: Implementation Phases

To transition this blueprint to the codebase via Antigravity, we will execute in three strict phases to isolate the beta-level risks of the Quack protocol.

Phase 1: Quack Infrastructure & Auth

Deploy the central DuckDB server. Implement the basic ``quack_authentication_function`` or configure a reverse proxy (Envoy/Nginx) if network isolation is required.

Phase 2: The Dagster Execution Engine

Wire the Dagster assets. Implement the routing logic that discriminates between the Tier 1 (Ollama) and Tier 2 (External API) execution paths.

Phase 3: Target Containerization & Telemetry

Containerize the target multi-agent topology. Implement the DuckDB client inside the worker nodes using the ``ATTACH 'quack:central-server'`` command to stream logs.